

Architecture et Programmation Web

Séance 5 : Modélisation de Données et SQL

Nicolas Burny

27 mars 2026

ECAM

Introduction

Pourquoi une base de données ?

- **Persistence** : Les données ne s'effacent pas à l'arrêt du programme.
- **Structure** : Les données sont organisées de manière rigide et prévisible.
- **Relations** : Capacité à lier des informations complexes (Auteur → Post).
- **Performance** : Indexation pour des recherches rapides.

SQLite

SQLite est un moteur de base de données relationnelle léger, sans serveur, qui stocke l'intégralité de la base dans un seul fichier. Idéal pour le développement et les applications embarquées.

Structure des Tables

Table : users

id (PK)	username	email
1	nicolas	n@ecam.be
2	julie	j@test.com

- **PK (Primary Key)** : L'identifiant unique.
- **Colonnes** : Définissent le type de donnée (TEXT, INTEGER, DATE).
- **Lignes** : Les données réelles (instances).

Comment lier une Feature request à son User ?

- On ajoute une **FK (Foreign Key)** dans la table "enfant".
- La valeur de la FK correspond à une PK de la table "parent".

`features.user_id → users.id`

Exemple : Clés Étrangères

Table : users

id (PK)	username
1	nicolas
2	julie

Table : features

id	title	user_id
101	Dark Mode	1
102	Export PDF	1
103	API Access	2

Le Modèle Entité-Association (ER)

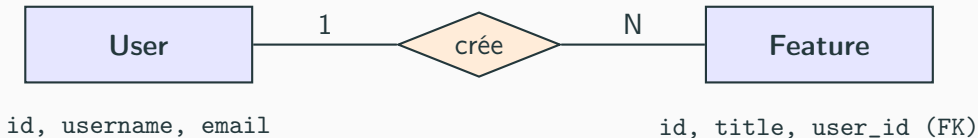
Avant de créer des tables, on définit le **Modèle Conceptuel de Données (MCD)**.

- **Entité** : Un objet ou concept du monde réel (ex : Utilisateur, Produit).
- **Attribut** : Une propriété d'une entité (ex : nom, prix).
- **Association** : Un lien logique entre deux entités (ex : un utilisateur *achète* un produit).

La **cardinalité** définit le nombre d'occurrences d'une entité qui peuvent être liées à une occurrence d'une autre entité.

- **One-to-One (1 :1)** : Un utilisateur a un seul profil (et inversement).
- **One-to-Many (1 :N)** : Un client a plusieurs factures, mais une facture appartient à un seul client.
- **Many-to-Many (N :M)** : Un étudiant suit plusieurs cours, et un cours est suivi par plusieurs étudiants.

Diagramme Entité-Association (ER)



- Les **rectangles** représentent les entités.
- Le **losange** représente l'association.
- Les **chiffres** sur les arêtes indiquent les cardinalités.

Implémentation : du 1 :N aux Tables

Pour une relation 1 :N, on place une **Clé Étrangère (FK)** dans la table du côté "N".

Exemple : Un Utilisateur (1) crée plusieurs Features (N)

`features.user_id (FK) → users.id (PK)`

Exemple SQL : Relation 1 :N

```
CREATE TABLE users (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    username TEXT NOT NULL  
);
```

```
CREATE TABLE features (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    title TEXT NOT NULL,  
    user_id INTEGER,  
    FOREIGN KEY (user_id) REFERENCES users(id)  
);
```

Implémentation : Many-to-Many (N :M)

Pour une relation N :M, on utilise une **Table de Jonction** (ou table pivot).

```
CREATE TABLE user_groups (  
    user_id INTEGER,  
    group_id INTEGER,  
    PRIMARY KEY(user_id, group_id),  
    FOREIGN KEY(user_id) REFERENCES users(id),  
    FOREIGN KEY(group_id) REFERENCES groups(id)  
);
```

Le Langage SQL

Création de structure (DDL)

```
CREATE TABLE users (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    username TEXT NOT NULL UNIQUE,  
    email TEXT NOT NULL  
);
```

```
CREATE TABLE features (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    title TEXT NOT NULL,  
    created_at DATE,  
    user_id INTEGER,  
    FOREIGN KEY(user_id) REFERENCES users(id)  
);
```

Sélectionner des données (SELECT)

-- Tout récupérer

```
SELECT * FROM features;
```

-- Filtrer et trier

```
SELECT title FROM features  
WHERE priority = 'Haute'  
ORDER BY title ASC;
```

Modifier les données (CRUD)

```
INSERT INSERT INTO users (username) VALUES ('Bob');
```

```
UPDATE UPDATE features SET status = 'Done' WHERE id = 5;
```

```
DELETE DELETE FROM features WHERE id = 12;
```

Attention

Toujours utiliser un WHERE avec UPDATE et DELETE !

Jointures

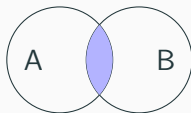
Lier les tables (JOIN)

```
SELECT features.title, users.username  
FROM features  
JOIN users ON features.user_id = users.id;
```

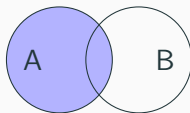
- JOIN (ou INNER JOIN) ne retourne que les lignes qui ont une correspondance dans les deux tables.
- Permet de reconstruire l'information complète à partir de données normalisées.

Les différents types de Jointures

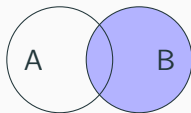
INNER JOIN



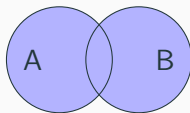
LEFT JOIN



RIGHT JOIN



FULL JOIN



SQLite

SQLite ne supporte pas `RIGHT JOIN` ni `FULL OUTER JOIN`. On peut les simuler en inversant l'ordre des tables ou en combinant des `LEFT JOIN` avec `UNION`.

Fonctions d'Agrégation

-- Nombre total de features

```
SELECT COUNT(*) FROM features;
```

-- Nombre de features par utilisateur

```
SELECT users.username, COUNT(features.id) AS nb_features  
FROM users  
LEFT JOIN features ON users.id = features.user_id  
GROUP BY users.id;
```

-- Filtrer les groupes (HAVING)

```
SELECT user_id, COUNT(*) AS nb  
FROM features  
GROUP BY user_id  
HAVING nb > 3;
```

Index

Accélérer les recherches avec les Index

Un **index** est une structure de données qui accélère les recherches sur une colonne.

```
-- Créer un index sur une colonne souvent filtrée  
CREATE INDEX idx_features_user ON features(user_id);
```

- Les **Primary Keys** sont automatiquement indexées.
- Utiliser un index sur les colonnes utilisées dans WHERE, JOIN, ORDER BY.
- Un index accélère la lecture mais ralentit légèrement les écritures.

Transactions SQL

Garantir la cohérence des données

Une **transaction** regroupe plusieurs opérations en une unité atomique : tout réussit, ou tout est annulé.

```
BEGIN TRANSACTION;
```

```
INSERT INTO users (username, email)
    VALUES ('alice', 'alice@ecam.be');
INSERT INTO features (title, user_id)
    VALUES ('Export CSV', last_insert_rowid());
```

```
COMMIT; -- Valide les deux insertions
```

```
-- En cas d'erreur :
```

```
ROLLBACK; -- Annule tout depuis le BEGIN
```

T-SQL (SQL Server)

```
BEGIN TRY
    BEGIN TRANSACTION;
    -- Vos requêtes SQL
    COMMIT;
END TRY
BEGIN CATCH
    ROLLBACK;
END CATCH
```

SQLite

Ce n'est pas possible en **SQLite** ! Le moteur ne supporte pas de blocs TRY...CATCH.
La gestion d'erreur doit se faire dans le code applicatif (ex : Python).

Conclusion et Récapitulatif

- **Entité** → Table → Classe (en ORM)
- **Cardinalité 1 :N** → Clé étrangère dans la table "N"
- **Cardinalité N :M** → Table de jonction
- **CRUD** : SELECT, INSERT, UPDATE, DELETE
- **JOIN** : Reconstruire les données normalisées
- **Agrégation** : COUNT, SUM, GROUP BY
- **Transactions** : Garantir la cohérence

Questions ?